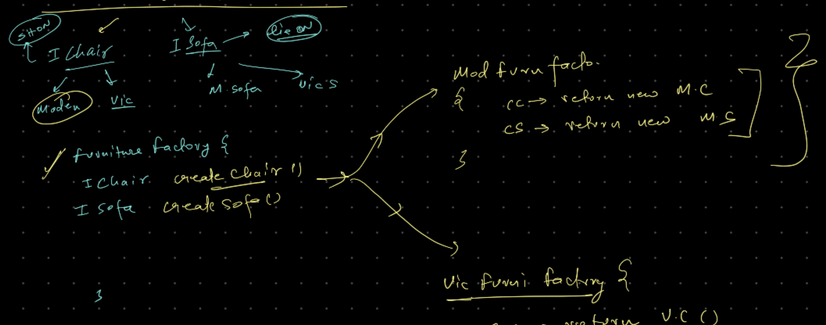
Astract Factory Design Pattern



Sitton — I Chair — Modern — Vic

I Sofa — Lieon — M sofa — Vic's

furniture factory {
I Chair create Chair ()
I sofa create sofa ()
}

Mod furn facto.
{ cc → return new M.C
cs → return new M.S
}

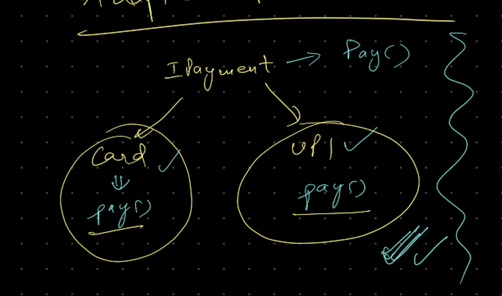
Vic furni factory {
c.c → return V.C ()
c.s → return V.S ()
}

Adapter Pattern

IPayment → Pay()

Card ⇓ pay()

UPI pay()



Processor {
IPayment ip; ←

process () {
sout →
ip pay (); →]
sout →
}
}

Cash Payment {
makepayment () {
complicated
}
}

Goal: → make cash payment class adaptive to the processor without changing anything in Cash Payment.

Cash Payment Adapter → Ipayment {

Cash Payment cp;
Cash P Adapter (Cash Payment cp) cp → cp

pay () {
cp. make payment ()
}
}

{ IPayment → pay() } I

{ Card ↳ @ over pay } c_1

{ UPI @ overid pay } c_2

(3) (X) 6 new classes

uses C → (1) Fee Deduction { (2) Reward { (3) Both Fee & Reward

abstract DecoraterBase {
Ipayment ip
abstract proccess () X
}

Fee Ded Decorate {
process () {
Feed
ip. pay
}
}

Reward Decora {
process {
ip pay
Reward
}
}

Both Decora {
process {
Fee
ip pay
Reward
}
}